Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №53

ТЕМА: Валідація даних у сучасному GUI та обробка помилок введення з виведенням повідомлень користувачу.

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- ознайомитися з поняттям валідації даних у графічному інтерфейсі;
- навчитися перевіряти коректність введених користувачем даних перед їх обробкою;
- вивчити способи наочного інформування користувача про помилку введення;
- розвинути навички поєднання try...except із логікою графічного інтерфейсу.

Теоретичні відомості

1. Навіщо потрібна валідація даних у GUI

Валідація даних — це перевірка того, що введена користувачем інформація відповідає очікуваному формату й обмеженням, перш ніж програма використає її для подальших обчислень. У консольних програмах некоректне введення часто просто призводило до аварійного завершення чи повторного запиту через input(). У графічному інтерфейсі користувач не бачить консоль, тому програма зобов'язана сама, наочно та зрозуміло, повідомити про помилку прямо у вікні — інакше користувач просто не зрозуміє, чому програма «не працює» після натискання кнопки.

2. Найпростіший спосіб: перевірка умовним оператором

Якщо вимоги до даних прості (наприклад, поле не повинно бути порожнім), достатньо звичайної перевірки if:

```
def submit():
    name = entry.get()
    if name == '':
        error_label.configure(text="Помилка: введіть ім'я")
    else:
        error_label.configure(text='')
        print('Прийнято ім\'я:', name)
```

Мітку для виведення помилки (error_label) зазвичай створюють одразу під полем введення й тримають порожньою, доки не виникне помилка — це не займає зайвого місця в інтерфейсі, коли все введено правильно, але одразу привертає увагу, щойно щось пішло не так.

3. Валідація числових даних через try...except

Коли очікується число, а користувач може ввести довільний текст, перетворення типу (int() чи float()) може викликати виняток ValueError. У консольній програмі ми обгортали такий код у try...except; у GUI-програмі принцип той самий, лише замість print() з повідомленням про помилку викликають configure() відповідної мітки:

```
def calculate():
    try:
        age = int(entry.get())
        if age < 0 or age > 120:
            error_label.configure(text='Вік має бути від 0 до 120')
            return
        error_label.configure(text='')
        result_label.configure(text=f'Через рік вам буде {age + 1}')
    except ValueError:
        error_label.configure(text='Помилка: вік має бути цілим
числом')
```

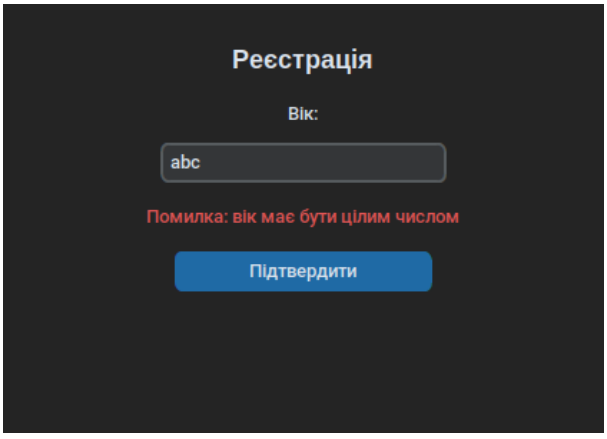


Рис. 1. Приклад повідомлення про помилку валідації, показаного прямо під полем введення

Команда return усередині except чи після невдалої перевірки важлива: вона негайно завершує виконання функції-обробника, не дозволяючи програмі продовжити роботу з завідомо некоректними даними далі за текстом функції.

4. Комбінована перевірка кількох полів

Коли форма містить кілька полів, зручно виконувати всі перевірки послідовно, збираючи (або одразу показуючи) перше знайдене порушення, а обчислення чи збереження даних виконувати лише тоді, коли всі перевірки пройдено успішно:

```
def register():
    name = name_entry.get()
    age_text = age_entry.get()
    if name.strip() == '':
        error_label.configure(text="Помилка: поле «Ім'я» не може
```

```
бути порожнім")
        return
    try:
        age = int(age_text)
    except ValueError:
        error_label.configure(text='Помилка: вік має бути числом')
        return
    if age < 6 or age > 100:
        error_label.configure(text='Помилка: вік поза допустимим
діапазоном')
        return
    error_label.configure(text='')
    print(f'Зареєстровано: {name}, {age} років')
```

5. Візуальні прийоми привернення уваги до помилки

Окрім тексту помилки, для підвищення наочності часто застосовують додаткові візуальні сигнали: червоний колір тексту помилки (text_color='#e05555' чи подібний відтінок), тимчасову зміну кольору рамки самого поля введення (border_color) на червоний, а після виправлення — повернення звичайного кольору. Такі прийоми допомагають користувачу відразу, без уважного читання тексту, зрозуміти, що саме поле містить помилку:

```
def validate_entry():
    try:
        int(age_entry.get())
        age_entry.configure(border_color='gray')
        error_label.configure(text='')
    except ValueError:
        age_entry.configure(border_color='#e05555')
        error_label.configure(text='Введіть ціле число')
```

Хід роботи

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Межі перевірок залежать від N (mod — остача від ділення):

Завдання	**Дані варіанта**	**Приклад N = 7**
3	діапазон кількості від N до 50 + N	від 7 до 57
4	мінімальний вік m = 14 + N mod 3	m = 15

б) Кожне завдання — окремий файл Прізвище_ПР53_зN.py (наприклад, Ivanenko_PR53_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Для кожного завдання додайте скріншот вікна із видимим заголовком. Імпорт — import customtkinter as ctk; програма завершується app.mainloop(). Тексти елементів і повідомлень — точно як в умові. Повідомлення про помилки показуйте в інтерфейсі (у мітці), а не в консолі.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР53_з1.py. Додайте CTkEntry для імені, кнопку «Перевірити» та мітку помилки, що з'являється (червоним), якщо поле порожнє (після strip()): «Введіть ім'я». За коректного імені мітка очищується.

   Приклад: порожнє поле → «Введіть ім'я»; «Олег» → мітка порожня.

2. Додайте CTkEntry для кількості товару; перевірку виконайте через try...except: для не-числа — «Введіть ціле число».

   Приклад: «abc» і «3.5» → «Введіть ціле число»; «12» → без помилки.

3. Додайте перевірку діапазону: кількість має бути від A до B вашого варіанта; інакше «Кількість має бути від A до B».

   Приклад (N = 7): A = 7, B = 57: «6» → «Кількість має бути від 7 до 57»; «7» → коректно; «58» → помилка діапазону.

4. Форма з полями «Ім'я» та «Вік» (вік — ціле від m до 100, m вашого варіанта): перевіряйте послідовно й показуйте лише перше порушення («Введіть ім'я», «Вік має бути числом», «Вік має бути від m до 100»); після помилки — return.

   Приклад (N = 7): m = 15: порожнє ім'я й вік «abc» → «Введіть ім'я»; «Олег» і «abc» → «Вік має бути числом»; «Олег» і «14» → «Вік має бути від 15 до 100».

5. Для одного з полів змінюйте border_color: червоний (#e74c3c) за некоректних даних і зелений (#2ecc71) за коректних.

   Приклад: після кожної перевірки колір рамки відповідає результату.

6. Завдання підвищеної складності*. Форма «розрахунок знижки»: сума — додатне число, відсоток — число від 0 до 100. Помилки: «Сума має бути додатним числом», «Знижка має бути числом від 0 до 100»; успіх — «До сплати: … грн» (два знаки).

   Приклад: 1000 і 15 → «До сплати: 850.00 грн»; -5 і 10 → «Сума має бути додатним числом»; 500 і 120 → «Знижка має бути числом від 0 до 100».

Критерії оцінювання

Складова	Оцінювання	Макс. балів
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, помилка виводиться в консоль, а не в інтерфейсі, показано кілька повідомлень замість першого, не перевірено порожнє поле, межі не відповідають варіанту) — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити межі перевірки чи текст повідомлення). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Чому в графічному інтерфейсі недостатньо просто вивести помилку в консоль, як у консольній програмі?
2. Яку конструкцію використовують для перехоплення помилки перетворення тексту в число?

3. Для чого в обробниках валідації часто використовують команду return одразу після виявлення помилки?
4. Які додаткові візуальні прийоми (окрім тексту) можна застосувати, щоб виділити поле з помилкою?
5. Як організувати перевірку форми з кількома полями так, щоб користувач бачив саме перше знайдене порушення?